



Módulo 3

Fundamentos de animación 2D

Preparado por: Ing. Samuel Jiménez



Unidad 1

Conceptos Fundamentales de la herramienta

Objetivos Generales

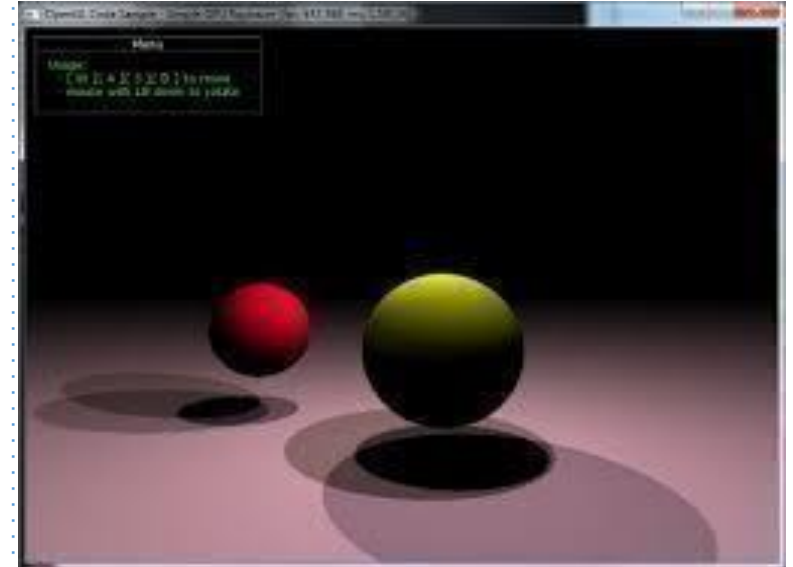
- ***Conocer los conceptos fundamentales de OpenGL para visualizar gráficos 2D y 3D.***
- ***Definir las funciones básicas presentes en OpenGL.***
- ***Conocer la estructura básica de un programa escrito en OpenGL***



Qué es OpenGL

OpenGL (Open Graphics Library) es una **API multiplataforma y de código abierto** diseñada para el desarrollo de aplicaciones gráficas 2D y 3D.

Proporciona un conjunto de funciones que **permiten comunicarse con la tarjeta gráfica (GPU)** para dibujar y manipular gráficos en tiempo real.



Como funciona OpenGL

OpenGL funciona como un intermediario entre el programador y la GPU.

- El programador envía instrucciones gráficas, entre las instrucciones tenemos:
 - ☐ Dibujar **primitivas gráficas** en 2D o 3D (puntos, líneas, polígonos)
 - ☐ **Transformación** de primitivas
 - ☐ **Atención a eventos de teclado, ratón y ventana**
 - ☐ Trazado de texturas
 - ☐ Definición de materiales
 - ☐ Iluminación y sombreado
 - ☐ **Menús desplegables**
 - ☐ Niebla, animación, entre otros efectos especiales



- OpenGL traduce esas instrucciones en comandos que entiende la tarjeta gráfica.
- La GPU procesa y renderiza la imagen final en la pantalla.

Prefijos en OpenGL

El prefijo **gl** es una función de la librería de OpenGL

El prefijo **glu** procede de la librería GLU

El prefijo **glut** son funciones de la librería GLUT
(Librería de utilidades para ventanas y eventos)

```
{gl, glu, glut} <un nombre> [{d, f, u, ... etc}] [v]
```

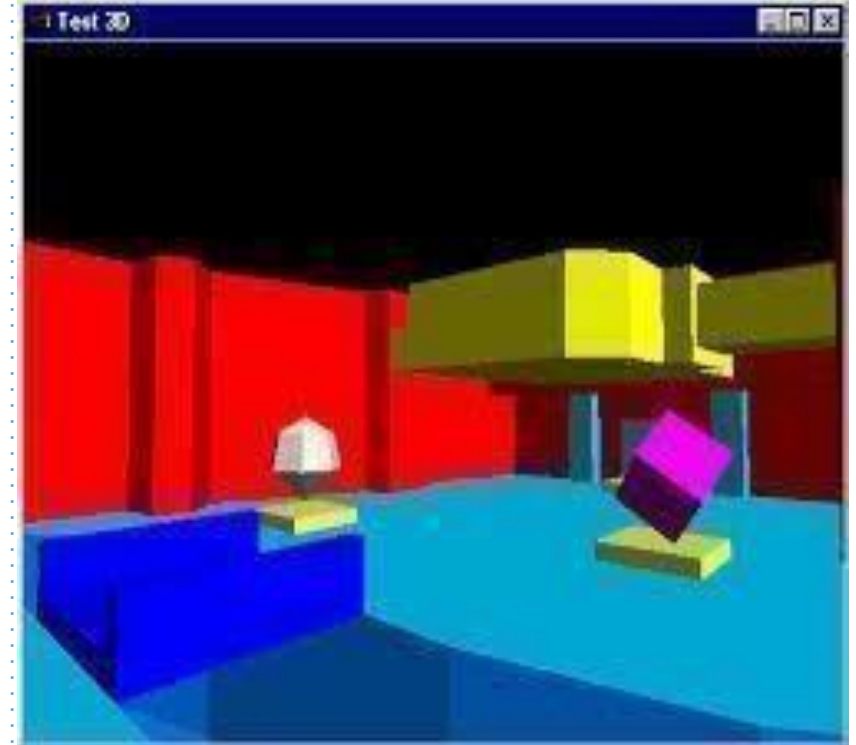
Ejemplos de funciones: gluPerspective, glMaterialf, glMaterialfv, glColor3f, glutSwapBuffers... **etc.**



Estructura de un programa OpenGL

Las principales acciones que se pueden realizar mediante órdenes OpenGL son las siguientes:

- ✓ Inicialización del sistema y de las ventanas
- ✓ Dibujo de la escena
- ✓ Bucle de gestión de eventos
- ✓ Redibujo de la escena



Estructura de un programa OpenGL

```
#include <GL/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT); // Limpia la pantalla
    glBegin(GL_TRIANGLES);       // Dibuja un triángulo
        glVertex2f(-0.5, -0.5);
        glVertex2f( 0.5, -0.5);
        glVertex2f( 0.0,  0.5);
    glEnd();
    glFlush();                   // Ejecuta los comandos
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(400, 300);
    glutCreateWindow("Mi primera ventana");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Manejo de Ventanas en OpenGL

Permiten la inicialización del sistema para que se abra una ventana gráfica para dibujar:

- ❑ `glutInit(int*argc, char **argv)`: Inicializa las utilidades de OpenGL
- ❑ `glutInitDisplayMode(unsigned int mode)`: Selecciona el modo de pantalla
Los modos que pueden utilizarse son:
 - ✓ `GLUT_SINGLE`: indica que se debe usar un solo buffer
 - ✓ `GLUT_RGB`: el tipo de modelo de color con el que se dibujará
 - ✓ `GLUT_DOUBLE`: Se utiliza para animaciones
 - ✓ `GLUT_DEPTH`: especifica un búfer de 32 bits de profundidad
 - ✓ `GLUT_ALPHA`: especifica un búfer de transparencia de destino
 - ✓ `GLUT_ACCUM`: especifica un búfer de acumulación

Manejo de Ventanas en OpenGL

- ❑ `glutInitWindowPosition(intx,inty)`: Posiciona la ventana en la pantalla
- ❑ `glutInitWindowSize(intwidth,intsize)`: Selecciona el tamaño de la ventana
- ❑ `glutCreateWindow (char*cadena)`: Abre la ventana, superámetro es el nombre o título de la ventana.

Funciones de Visualización OpenGL

El grupo de funciones de visualización definen la función que se llama para visualizar la escena.

Las ordenes que forman parte de este grupo son:

- `glutDisplayFunc (void(*func)(void))`: especifica la función que dibuja la escena
- `glutPostRedisplay (void)`: Actualiza la ventana actual
- `glClearColor(r, g, b, a)` → Color de fondo.
- `glMatrixMode(GL_PROJECTION)` → Define la proyección.
- `gluOrtho2D(xmin, xmax, ymin, ymax)` → Define coordenadas 2D.

Ejecución del programa OpenGL

La función de este grupo comienza:

1. La ejecución del programa
2. Visualización de todas las ventanas creadas
3. Permite el envío de ordenes de dibujo
4. Gestiona todos los eventos que se generen

`glutMainLoop()`: desencadena la ejecución del programa.

Gestión de eventos en OpenGL

El grupo de funciones de la gestión de eventos especifican las rutinas que se ejecutan cuando se produce un evento determinado.

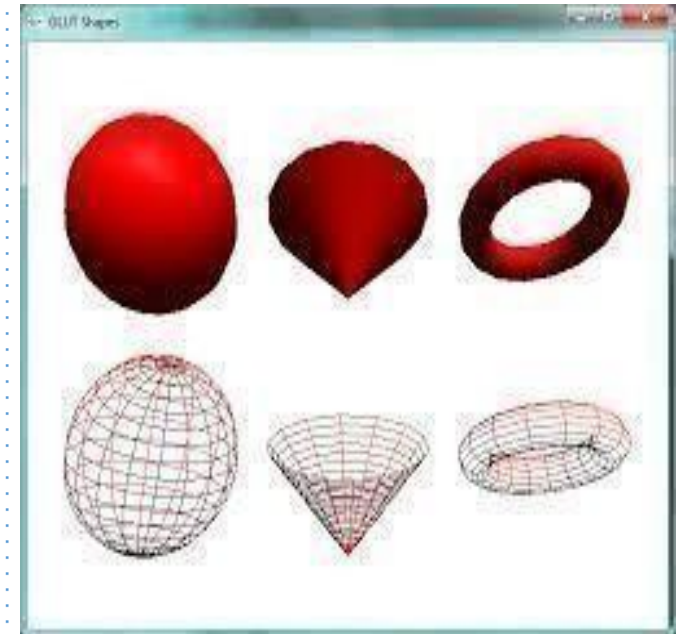
- `glutReshapeFunc`: gestión de eventos de redimensionamiento de la ventana.
- `glutKeyboardFunc`: gestión de eventos del teclado. Funciona cuando se pulsa una tecla.
- `glutMouseFunc`: gestión de eventos del ratón. Funciona cuando se pulsa un botón del ratón.
- `glutMotionFunc`: gestión de eventos del ratón. Funciona cuando se mueve el ratón con un botón pulsado.
 - `glutIdleFunc(funcion)` → Función en reposo

Funciones primitivas OpenGL

Este grupo permite visualizar las primitivas de objetos predefinidos de dibujo mas complejas que las que incluyen OpenGL, como la esfera, tetera, cono, entre otras.

Las funciones son las siguientes:

- ❖ `glutWireCone(base,height,slices,stacks)`
- ❖ `glutSolidCone (base,height,slices,stacks)`
- ❖ `glutWireCube (size)`
- ❖ `glutSolidCube (size)`
- ❖ `glutWireDodecahedron(void)`
- ❖ `glutSolidDodecahedron (void)`
- ❖ `glutWireicosahedron(void)`
- ❖ `glutSolidicosahedron (void)`

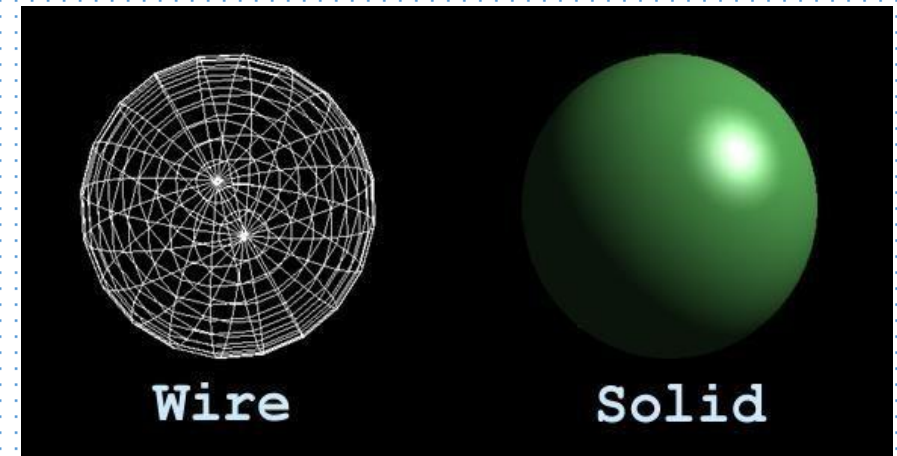


Funciones primitivas OpenGL

- ❖ glutWireOctahedron (void)
- ❖ glutSolidOctahedron (void)
- ❖ glutWireSphere (radius, slices, stacks)
- ❖ glutSolidSphere (radius, slices, stacks)
- ❖ glutWireTeapot (void)
- ❖ glutSolidTeapot (void)
- ❖ glutWireTetrahedron (void)
- ❖ glutSolidTetrahedron (void)

```
glut{Wire | Solid}{Sphere, Cube, ... etc}(<parámetros>)
```

```
glutWireSphere(radius, slices, stacks)  
glutSolidTeapot(size)
```



Funciones primitivas OpenGL

- GL_POINTS → Puntos.
- GL_LINES → Líneas entre pares de vértices.
- GL_LINE_STRIP → Líneas conectadas.
- GL_TRIANGLES → Triángulos independientes.
- GL_QUADS → Cuadriláteros.
- GL_POLYGON → Polígonos cerrados.

```
glBegin(GL_TRIANGLES);  
    glVertex2f(0.0, 0.0);  
    glVertex2f(1.0, 0.0);  
    glVertex2f(0.5, 1.0);  
glEnd();
```

Orden en OpenGL

- ❖ Dentro de una orden OpenGL se maneja un conjunto de prefijos y sufijos que ayudan a identificar el tipo de datos o número de parámetros que se estará trabajando.

- ❖ **Ejemplo:**

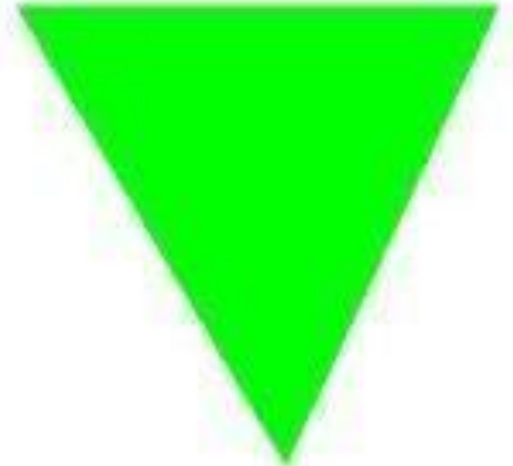
gl: Función de la librería OpenGL glut

Vertex: comando raíz

2: Cantidad de argumentos enviados

f: Tipo de argumento flotante

```
glColor3f(0.0,1.0,0.0);  
glVertex2f(0.0,-0.4);
```



Colores en OpenGL

`glClearColor(0.0,0.0,0.0,1.0)` En **OpenGL**, el **color** se maneja mediante funciones que especifican los valores de **rojo (R)**, **verde (G)**, **azul (B)** y, en algunos casos, el **canal alfa (A)** para transparencia.

El Rango de la mezcla de los componentes de color varia entre 0.0 y 1.0).

Color	Rojo	Verde	Azul
Amarillo	1.0	1.0	0.0
Azul	0.0	0.0	1.0
Blanco	1.0	1.0	1.0
Cian	0.0	1.0	1.0

`glColor3f(1.0, 0.0, 0.0);` → Rojo puro.

`glColor4f(0.0, 1.0, 0.0, 0.5);` → Verde semitransparente.

Colores en OpenGL

Color	Rojo	Verde	Azul
Gris claro	0.75	0.75	0.75
Gris Oscuro	0.25	0.25	0.25
Magenta	1.0	0.0	1.0
Marrón	0.60	0.40	0.12

Color	Rojo	Verde	Azul
Naranja calabaza	0.98	0.625	0.12
Negro	0.0	0.0	0.0
Purpura	0.60	0.40	0.70
Rojo	1.0	0.0	0.0

Colores en OpenGL

Color	Rojo	Verde	Azul
Rosa Pastel	0.98	0.04	0.70
Verde	0.0	1.0	0.0

Colores en OpenGL

```
#include <GL/glut.h>

void display() {
    glClearColor(0.0, 0.0, 0.0, 1.0); // Fondo negro
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_TRIANGLES);

        glColor3f(1.0, 0.0, 0.0); // Rojo
        glVertex2f(-0.5, -0.5);

        glColor3f(0.0, 1.0, 0.0); // Verde
        glVertex2f(0.5, -0.5);

        glColor3f(0.0, 0.0, 1.0); // Azul
        glVertex2f(0.0, 0.5);

    glEnd();
    glFlush();
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(400, 300);
    glutCreateWindow("Colores en OpenGL");
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Otras funciones de OpenGL

- ❖ `glFlush()`: es la encargada de realizar la limpieza y el borrado de los búferes de comando OpenGL. Este comando hace que se ejecute cualquier comando que este esperando a ejecutarse.
- ❖ `glClear (GL_COLOR_BUFFER_BIT)`: borra un búfero una combinación determinada.

Los argumentos pueden ser:

Buffer	Nombre
Buffer de color	GL_COLOR_BUFFER_BIT
Buffer de fondo	GL_DEPTH_BUFFER_BIT
Buffer de acumulación	GL_ACCUM_BUFFER_BIT
Buffer de patrón	GL_STENCIL_BUFFER_BIT

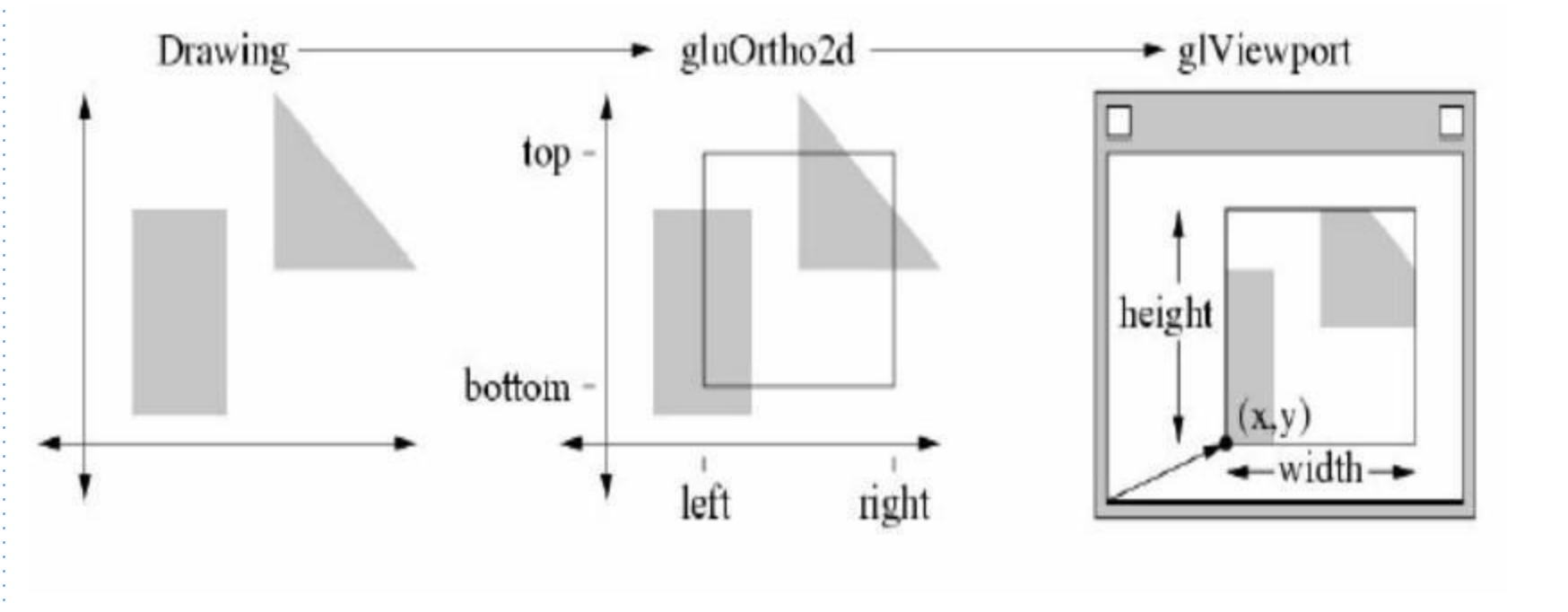
Proyección ortográfica

Es un tipo de proyección en la que los objetos **no se deforman por la perspectiva**:

- Las líneas paralelas en el mundo permanecen paralelas en la pantalla.
- El tamaño de los objetos no cambia con la distancia (no hay punto de fuga).
- Muy usada en **gráficos 2D**, interfaces y ciertos juegos 3D (vista isométrica).

Proyección ortográfica

- ❖ gluOrtho2D: Especifica las coordenadas de la región 2D que se proyectara en la ventana



`gluOrtho2D(xmin, xmax, ymin, ymax);`

`gluOrtho2D(0, 200, 0, 150);` → Mundo de 200×150 unidades.

Proyección ortográfica

```
#include <GL/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_POLYGON);    // Dibujar cuadrado
        glVertex2f(50, 50);
        glVertex2f(150, 50);
        glVertex2f(150, 150);
        glVertex2f(50, 150);
    glEnd();

    glFlush();
}
```

```
void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // Fondo blanco
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 200, 0, 200);        // Proyección ortográfica 2D
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(400, 400);
    glutCreateWindow("Proyeccion Ortografica");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

La ventana es de 400×400 píxeles.

El mundo lógico es de 200×200 unidades.

El cuadrado dibujado de (50,50) a (150,150) ocupa un espacio en coordenadas lógicas, que OpenGL escala a la pantalla.

gluOrtho2D en OpenGL – Guía Visual

`gluOrtho2D` define un sistema de coordenadas 2D. Es como elegir una “ventana” del mundo que quieres ver en la pantalla.

```
gluOrtho2D( left, right, bottom, top);
```

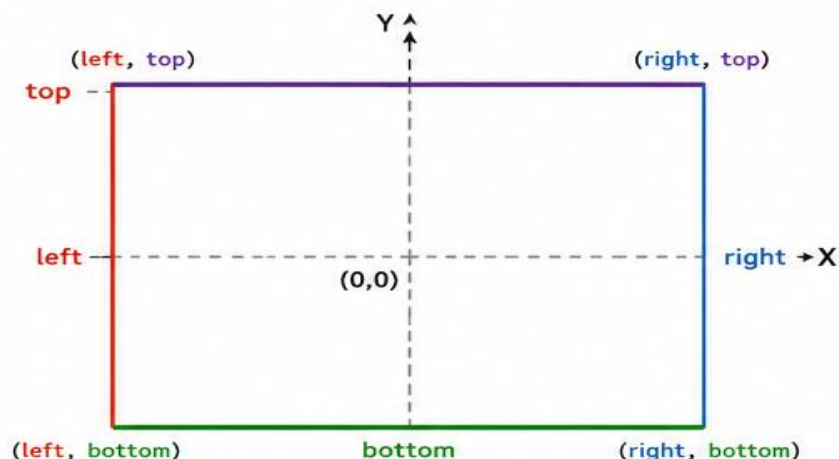
left : límite izquierdo
right : límite derecho
bottom : límite inferior
top : límite superior



IDEA CLAVE

No mueve los objetos.
Cambia **cómo los ves**.

1. SISTEMA DE COORDENADAS



2. EJEMPLO BÁSICO

```
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
gluOrtho2D(0, 100, 0, 100); // ventana de 0 a 100
```

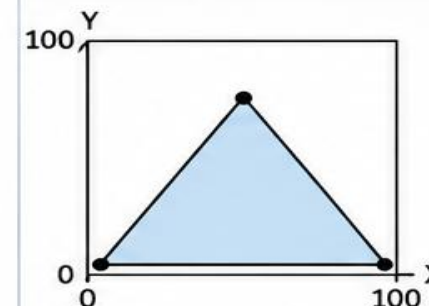


DIBUJAMOS UN TRIÁNGULO

```
glBegin(GL_TRIANGLES);  
glVertex2f(10, 10); // cerca de (0,0)  
glVertex2f(80, 10); // cerca de (100,0)  
glVertex2f(50, 70); // arriba  
glEnd();
```

¿QUÉ VEREMOS?

Un triángulo dentro de la ventana (0,0) a (100,100).

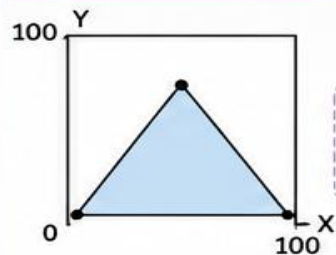


3. ZOOM (MISMA ESCENA, DISTINTA VENTANA)

Si cambiamos los valores de `gluOrtho2D`, vemos otra parte del mundo o hacemos zoom.

VISTA NORMAL

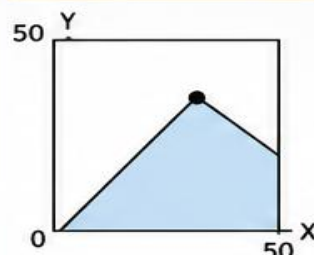
```
gluOrtho2D(0, 100, 0, 100)
```



Vemos todo el mundo de (0,0) a (100,100).

ZOOM (ACERCAMOS)

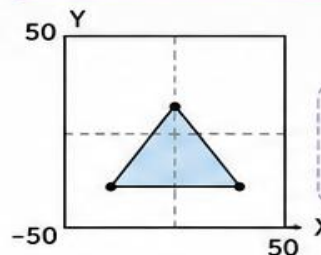
```
gluOrtho2D(0, 50, 0, 50)
```



Vemos solo de (0,0) a (50,50). El triángulo se ve más grande.

CENTRADO EN EL ORIGEN

```
gluOrtho2D(-50, 50, -50, 50)
```



El (0,0) queda en el centro de la ventana.

4. RESUMEN RÁPIDO

- ✓ Define el sistema de coordenadas 2D.
- ✓ Actúa como una “cámara ortográfica”.
- ✓ Permite hacer zoom y mover la vista.
- ✓ No altera los objetos, solo la vista.

5. ANALOGÍA FÁCIL



Todo el mapa
(0 a 100)



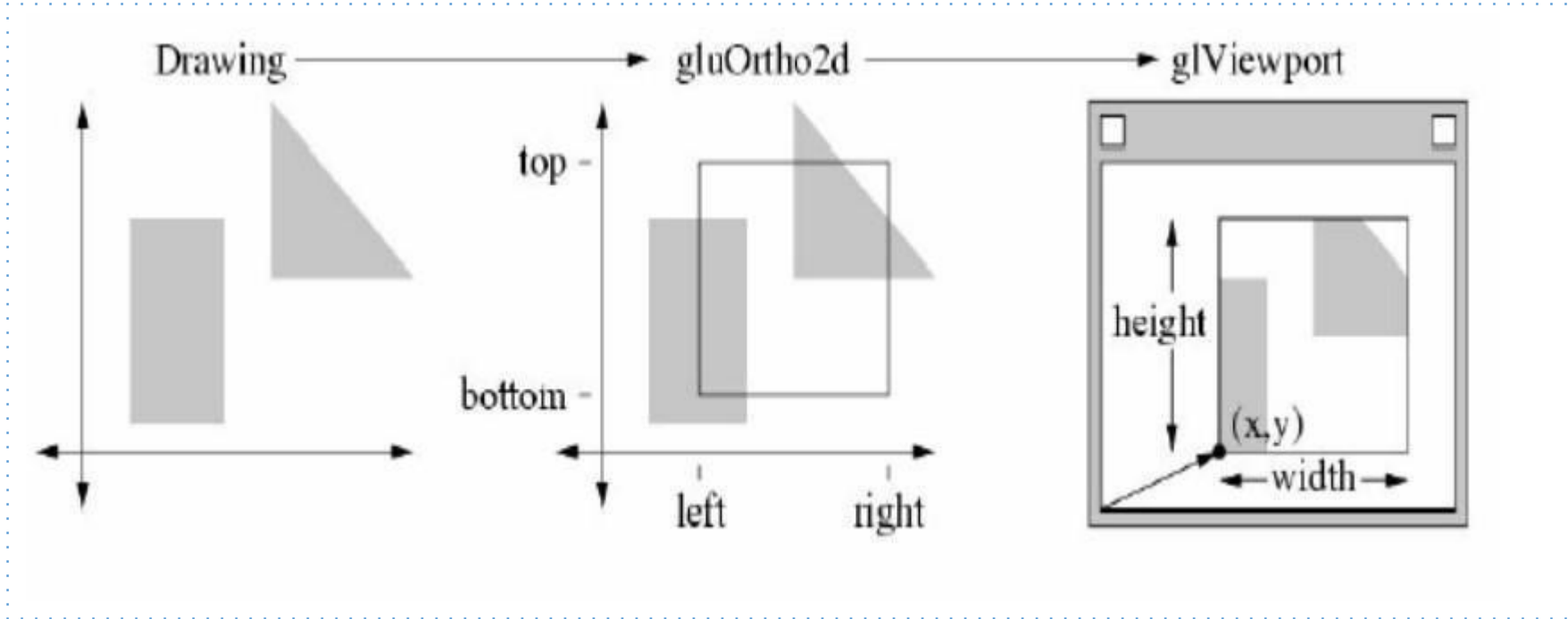
Zoom a una parte
(0 a 50)



Centro en el origen
(-50 a 50)

Proyección ortográfica

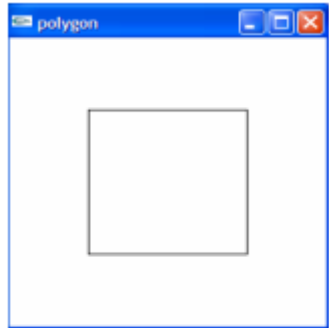
- ❖ El viewport es el área rectangular de la ventana en pantalla (en píxeles) donde OpenGL dibuja la escena. En otras palabras: si tu ventana mide 800×600 píxeles, el viewport define qué parte de esa ventana se usará para mostrar tu mundo lógico.



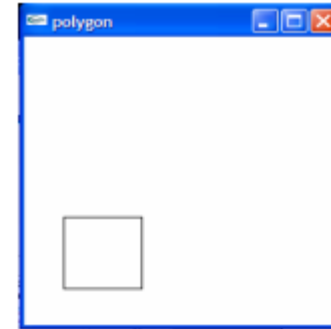
`glViewport(x, y, ancho, alto);`

Proyección ortográfica

❖ Ejemplo de Viewport



```
glViewport(0, 0, w, h);
```



```
glViewport(0, 0, w/2, h/2);
```

```
glViewport(x, y, ancho, alto);
```

Proyección ortográfica

❖ Ejemplo de Viewport

```
#include <GL/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Triángulo rojo (ocupa todo el viewport)
    glColor3f(1.0, 0.0, 0.0);
    glBegin(GL_TRIANGLES);
        glVertex2f(-0.5, -0.5);
        glVertex2f( 0.5, -0.5);
        glVertex2f( 0.0,  0.5);
    glEnd();

    glFlush();
}
```

```
void reshape(int w, int h) {
    glViewport(0, 0, w, h);           // Ajusta el viewport al tamaño de ventana
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1, 1, -1, 1);        // Proyección ortográfica
    glMatrixMode(GL_MODELVIEW);
}
```

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(400, 300);
    glutCreateWindow("Ejemplo de Viewport");
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);        // Llamada al cambiar tamaño de ventana
    glutMainLoop();
    return 0;
}
```

glViewport en OpenGL – Guía Visual

glViewport define la parte de la ventana (pantalla) donde se mostrará la imagen renderizada.

En otras palabras: recorta o posiciona la vista en la ventana.

```
glViewport( x, y, width, height );
```

x : posición horizontal del origen del viewport (en píxeles)
y : posición vertical del origen del viewport (en píxeles)
width : ancho del viewport (en píxeles)
height : alto del viewport (en píxeles)



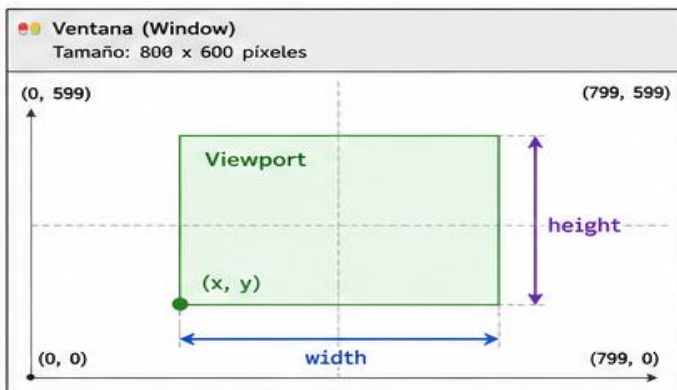
IDEA CLAVE

No cambia el sistema de coordenadas.
Solo cambia **DÓNDE** (y de qué tamaño)
se muestra la escena en la ventana.

Es como elegir "la pantalla dentro de la ventana".

1. ¿QUÉ ES EL VIEWPORT?

Imagina que tu ventana (pantalla) es un lienzo.
El viewport es un rectángulo dentro de ese lienzo
donde se dibuja todo.



El origen (0,0) de la ventana está en la esquina inferior izquierda.

2. EJEMPLO BÁSICO

Ventana creada de 800x600.

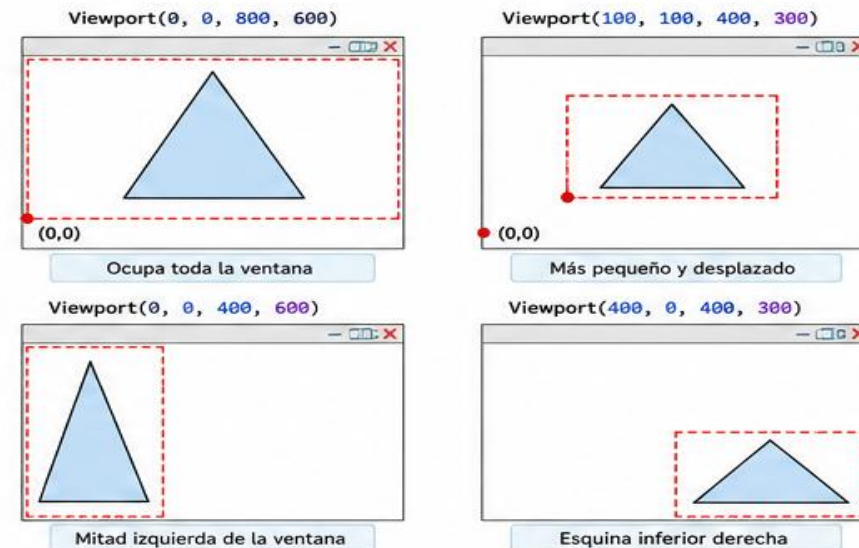
Mostraremos un triángulo usando diferentes viewports.

Código (ejemplo con OpenGL tradicional)

```
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
gluOrtho2D(0, 100, 0, 100); // Sistema de coordenadas 2D  
  
glClear(GL_COLOR_BUFFER_BIT);  
  
// Viewport: x, y, width, height  
glViewport(100, 100, 400, 300); // Dibuja en esa región  
  
// Dibujamos un triángulo (coordenadas de 0 a 100)  
glBegin(GL_TRIANGLES);  
glVertex2f(10, 10);  
glVertex2f(90, 10);  
glVertex2f(50, 90);  
glEnd();
```

3. RESULTADO VISUAL

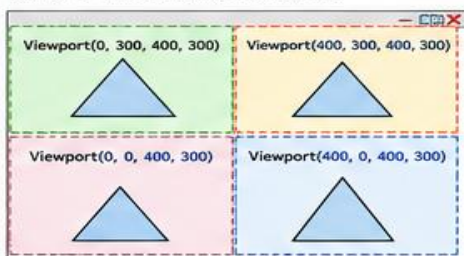
Mismo triángulo, distintos viewports en la misma ventana (800x600).



4. VARIOS VIEWPORTS (DIVIDIR PANTALLA)

Podemos dividir la ventana en varias regiones para mostrar diferentes vistas.

Ejemplo: 4 viewports (cuadrantes).



```
// Superior Izquierdo  
glViewport(0, 300, 400, 300);  
  
// Superior Derecho  
glViewport(400, 300, 400, 300);  
  
// Inferior Izquierdo  
glViewport(0, 0, 400, 300);  
  
// Inferior Derecho  
glViewport(400, 0, 400, 300);
```

5. IMPORTANTE: ORDEN DE LLAMADAS

El viewport debe configurarse después de crear la ventana y antes de dibujar.



6. PUNTOS CLAVE

- ✓ glViewport usa coordenadas en **PÍXELES**.
- ✓ (0,0) está en la esquina inferior izquierda de la ventana.
- ✓ No afecta las coordenadas de tu escena (eso lo hace la proyección).
- ✓ Puedes cambiar el viewport en cualquier momento.
- ✓ Útil para mini-mapas, HUD, vistas múltiples, editores, etc.

7. RESUMEN RÁPIDO



glViewport decide **DÓNDE** y de qué tamaño se mostrará tu escena.

Diferencia entre Viewport y Ortho

Imagina que tienes un mapa:

- **Proyección ortográfica** = el área del mapa que quieres ver (ejemplo: calles de una ciudad).

- **Viewport** = el tamaño del papel (o pantalla) donde lo imprimes (ejemplo: hoja A4 o A3).

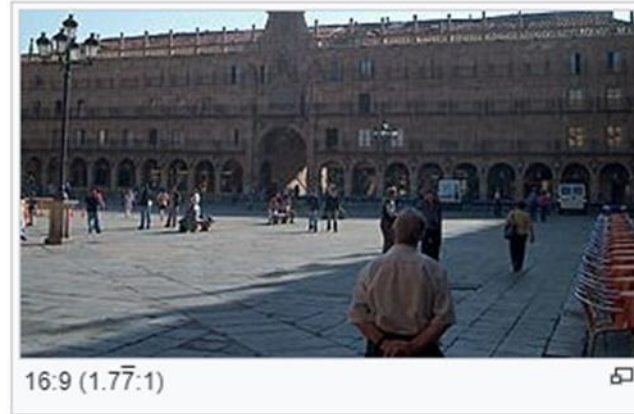
```
gluOrtho2D(0, 200, 0, 150);
```

El mundo lógico tiene coordenadas de **0 a 200 en X** y de **0 a 150 en Y**.

```
glViewport(0, 0, 400, 300);
```

Eso significa que todo el mundo lógico (200×150) se dibujará en un área de **400×300 píxeles** en la ventana.

Relaciones de aspectos



Comparación de varias relaciones de aspecto

	4:3	
	3:2	
	16:9	

La relación dimensional, a menudo llamada **relación de aspecto de una imagen**, es la proporción entre su ancho y su altura.

Cuando defines una proyección (ya sea ortográfica o en perspectiva), debes asegurarte de que la relación de aspecto de tu ventana lógica coincida con la de tu viewport/ventana en píxeles.

Ejemplo: 1024*768 4:3 (1024/768=1.33)
 1920*1080 16:9 (1920/1080=1.77)

Algunas relaciones de aspecto comunes son 4:3 y 16:9.

Una relación de aspecto de 4:3 significa que por cada cuatro unidades de ancho, hay tres unidades de alto.

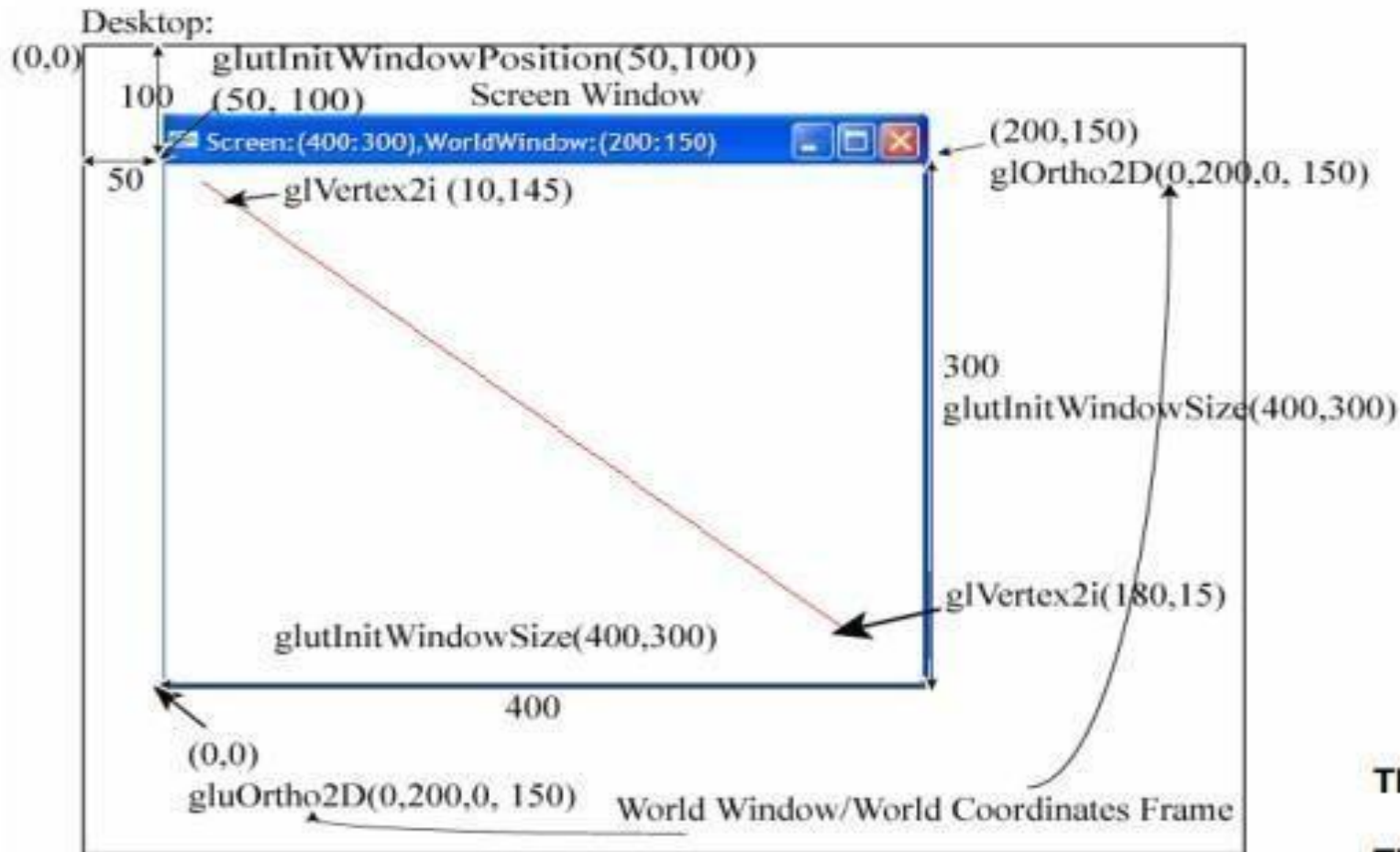
Si toma la relación de aspecto de 4:3 y divide 4 en 3, el resultado es 1.33.

4:3 → Pantallas clásicas (800×600, 1024×768).

16:9 → Pantallas actuales (1920×1080, 1280×720).

1:1 → Proporción cuadrada.

Como dibujar una línea utilizando las proyecciones



supongamos que tienes:

Ventana en píxeles = $400 \times 300 \rightarrow$ relación = $4/3$.

Proyección ortográfica = `glOrtho2D(0, 200, 0, 150)` $\rightarrow$ relación = $200/150 = 4/3$.

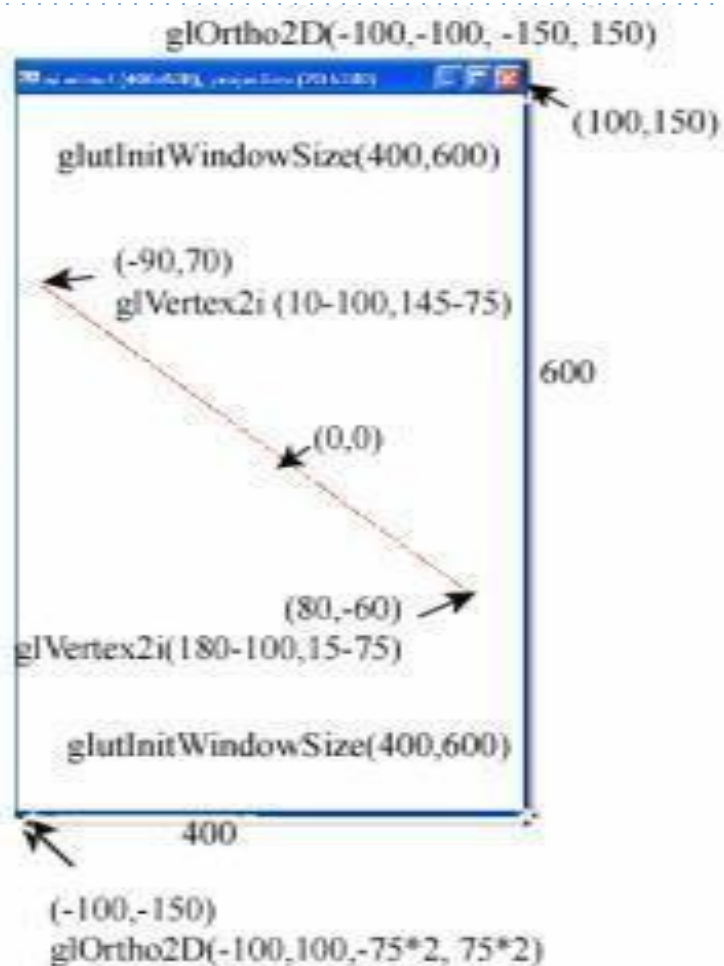
The aspect ratio of Screen Window:

$400:300 = 4:3$

The aspect ratio of World Window:

$200:150 = 4:3$

Como dibujar una línea utilizando las proyecciones



Aspect ratio of Screen Window: $400:600=2:3$

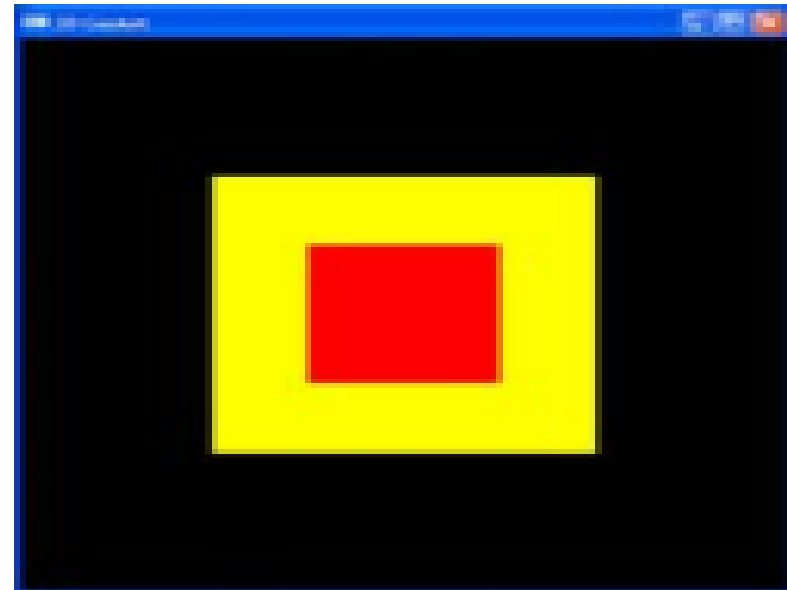
Aspect ratio of World Window

(orthogonal projection):

$(100-(-100)):(150-(-150))=2:3$

Práctica en clase

Logre realizar el siguiente cuadro e indique la relación de aspecto de su pc



¡GRACIAS!

